StateGuard: Failure Propagation and State-Consistent Recovery in Tool-Using Multi-Agent Workflows

Xinying Liu Jingwen Zhang Haodi Yuan
University of California San Diego, CSE 291A — Option 5
xil277@ucsd.edu jiz333@ucsd.edu hay101@ucsd.edu

Abstract. StateGuard studies failure propagation and state-consistent recovery in a tool-using deployment workflow. We implement a versioned runtime and record 99 real-model clean/fault pairs on AWS. Among 30 case/seed combinations valid under all three policies, task completion is 30% without recovery, 100% after full restart, and 80% with verification. Restart approximately doubles tokens and latency on this workload. These Phase 1 measurements establish baselines for a later selective-recovery comparison.

1 Research question and workflow

An incorrect port can propagate from analysis into a plan and committed configuration. Replanning leaves the write behind; snapshot rollback may erase a later administrator update. Our questions are: **RQ1**, which message and state dependencies carry failures; **RQ2**, which work remains reusable after legitimate updates; and **RQ3**, when does selective recovery save more than its bookkeeping costs?

Consider a stale claim that an obsolete database port is valid. If the Planner acts on it, the Executor can commit an invalid API configuration while a worker scales independently and an administrator changes API replicas. Recovery must repair the port, preserve valid later writes, and recheck the shared CPU budget. The workload records result delivery, Planner consumption, and committed tool effects separately; a received message alone is not proof of propagation.

Config, Dependency, and Resource Agents analyze configuration, versioned knowledge, and capacity. A Planner consumes structured results; a controlled Executor modifies API, worker, and database state. Checkers enforce compatibility, resource budgets, deployment goals, and protected authentication/audit fields. Python, Pydantic, SQLite, a CLI runner, and pytest implement the system. A TritonAI adapter logs prompts, responses, tokens, timing, and retries; deterministic agents isolate runtime semantics.

2 State-consistent recovery design

The store maintains $S(k) = (x_k, v_k)$ for each field. Results carry parent IDs and read sets $R_r = \{(k, v)\}$. A typed graph connects results, actions, state versions, and external events through consumption, reads, triggers, and writes. Recording is automatic; Planner dependencies conservatively include all three analysis results rather than claiming exact semantic use.

Given faulty result f, invalidate $I(f) = \{f\} \cup \mathrm{Desc}(f)$. Reuse requires

$$r \notin I(f) \ \wedge \ \forall (k, v) \in R_r : \ v_k^{\text{current}} = v. \tag{1}$$

Missing provenance, tool effects, and resource constraints require further checks. Invalid analyses and the Planner are recomputed; eligible independent results are retained.

Writes carry an expected version and action ID. A SQLite transaction checks the version and commits state, ledger, and events together; identical retries return the recorded outcome after a simulated lost acknowledgement. Selective replay writes changed fields and blocks conflicting updates. Full restart rebuilds state and replays recorded external events in order with value/version preconditions. Irreversible effects remain explicit. Correct recovery requires task goals, invariants, preserved legitimate updates, and no new unauthorized effects. Local transactions do not guarantee rollback or exactly-once effects at remote services.

For a write to field k, the tool admits the action only when its supplied expected version equals the current v_k. An administrator write to that field invalidates the stale precondition; a write to an unrelated field need not. The final checker re-evaluates global constraints, so independent work is not automatically safe when it consumes a shared CPU budget. We classify a recovery as blocked or partial when a conflict or irreversible effect prevents the full objective.

The integrated suite passes **156 tests**, covering independent-result reuse, administrator updates, same-field conflicts, replay preconditions, and lost-ACK deduplication. Selective recovery is implemented and tested; its real-model comparison remains Phase 2 work.

3 Phase 1 experimental protocol

Baselines. No recovery (NR) continues the injected workflow. Full restart (FR) executes a fresh clean attempt. Always verify (AV) checks intermediate outputs against simulated truth with one retry. AV is an optimistic oracle, not an independent detector. Recovery receives harness-supplied fault information, separating detection from recovery.

Configuration. The model is TritonAI `api-glm-5.3-flash`, called from an AWS CPU instance with Python 3.12.3. Each API request allows three attempts, with a five-second minimum request interval. Temperature is unspecified (provider default); seed labels 7, 11, and 19 fix fault schedules, not model sampling. Source revision is

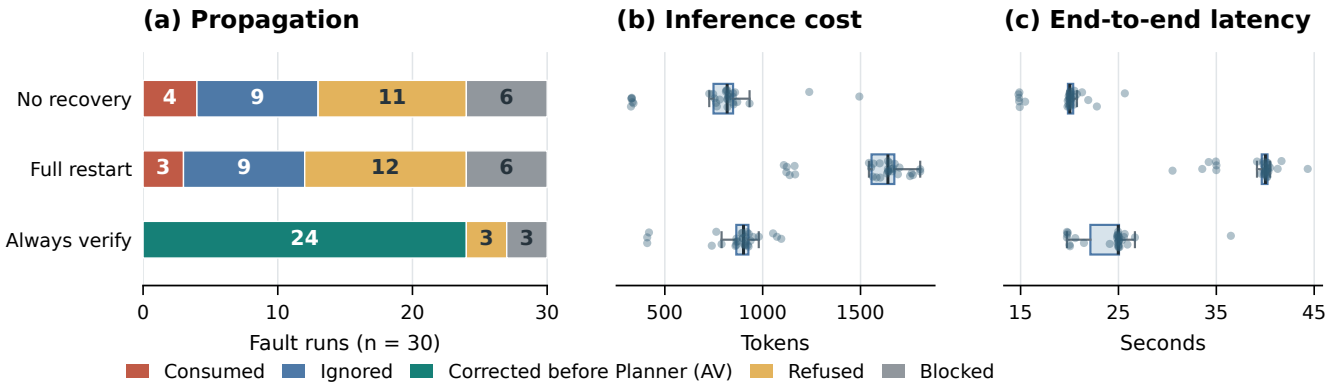


Figure 1: Matched fault runs (n = 30/policy). (a) Verdict after AV verification but before FR restart. (b–c) Fault-side tokens and latency include retries, restart, and pacing. Dots are runs; boxes show median/IQR.

`6930ac8`. Each pair has clean/fault runs; FR adds a restart. Manifests and paired traces retain configuration, outputs, and failures.

Workload. One deployment task and topology exercise 11 cases: three wrong-port variants, two stale-protocol variants, two analysis timeouts, two malicious protected-field instructions, and two intended ignored controls. Thus $11\times3\times3 = 99$ pairs span the three policies. Earlier eight-scenario smoke and paired pilots are separate because retry settings differ. Lost ACK and external-update recovery are tested deterministically, not exercised by this real-model matrix.

4 Measured results and failure analysis

We record whether the injected result was exposed to a downstream agent, whether its content was consumed, and whether a bad action committed. An ignored instruction therefore contributes to the verdict distribution but cannot establish that a recovery algorithm repaired a propagated fault. Clean/fault pairs share a workload and injection schedule; model sampling remains uncontrolled across separate calls.

Of 99 recorded pairs, 96 are valid: two FR pairs (seed 7) have HTTP 422 errors; one NR pair (seed 19) fails its clean control without an API error. Excluding these combinations from *every* policy leaves 9+11+10 = 30 matched comparisons. Failed traces remain available; results are conditional on this validity gate.

FR completes all matched tasks at 2.05× NR's tokens and 2.01× its latency. AV reaches 80% completion at 1.14× the tokens and 1.24× the latency. Figure 1 shows the per-run distributions; latency includes pacing, not just model compute.

NR has 4 consumed, 9 ignored, 11 refused, and 6 blocked injections. Under AV, the 24 raw "ignored" verdicts all follow a failed verification and a valid retry; the faulty outputs never reach Planner. Figure 1 therefore labels these *corrected before Planner*, retaining the raw verdicts in the traces. Under NR, all six adversarial runs succeed because the model ignores the injected instructions. Zero unauthorized commits does not establish recovery from an accepted attack.

Table 1: Matched fault-side outcomes. Recomputed stages exclude AV verification retries; calls, tokens, and latency include retries and FR restart.

Metric	NR	FR	AV
Completed tasks	9/30	30/30	24/30
Unauthorized commits	0	0	0
Mean tokens	756.7	1553.8	859.8
Mean latency (s)	19.46	39.04	24.19
Mean model calls	3.8	7.8	4.8
Mean recomputed stages	0	4	0

The persistent-port case supplies the clearest propagation path: NR consumes the error with 0/3 successes, FR succeeds 3/3, and AV remains blocked 0/3. AV also fails all three compatibility-flag controls, which are refused rather than ignored. Among accidental cases, completion is 3/24, 24/24, and 18/24 for NR, FR, and AV. Aggregate scores therefore conceal systematic case-level failures.

5 Remaining proposal and evaluation plan

This pilot covers one task/topology and constrained tools. Detection, tracking overhead, and selective-recovery performance remain unevaluated. Phase 2 will compare dependency-only and state-aware replay against FR and AV under identical fault identities, event schedules, and budgets. We target 24–36 tasks, adjusted after a cost pilot, with chain, fan-out, and diamond dependencies; unchanged state, unrelated updates, same-field conflicts, and shared-resource contention. We will separate development and evaluation workloads.

Appendix: Architecture, milestones, and references

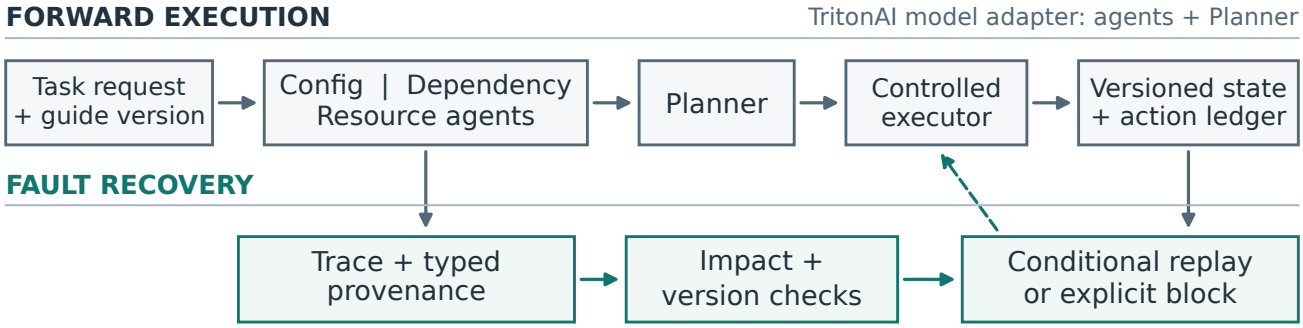


Figure 2: StateGuard runtime. The harness supplies the faulty result identity; replay follows provenance and field-version checks. The Phase 1 real-model matrix evaluates NR, FR, and AV, while selective replay is tested with deterministic tools.

Table 2: Project milestones (2026).

When	Deliverable and acceptance check
Nov. 3–15	Freeze artifacts; expand graph and event cases.
Nov. 16–25	Compare replay methods on matched faults.
Nov. 26–Dec. 4	Measure precision, conflicts, cost, and overhead.
Dec. 5–10	Reproduce results; submit code and final report.

Evidence boundary. Manifests, paired traces, checker outcomes, and analysis scripts reproduce the matched figures. The 156-test suite exercises controlled replay; the real-model matrix measures NR, FR, and AV. Detection, selective-replay performance, and arbitrary remote effects remain unmeasured.

Phase 2 evaluation. Chain, fan-out, and diamond workloads will test external updates and shared-state conflicts. Ablations remove field reads, version checks, and action deduplication. We will measure invalidation precision/recall, preserved work, blocked effects, tokens, latency, and tracking overhead.

Related work. AgentDojo [1], TAMAS [2], and A2ASecBench [3] evaluate agent security; our traces additionally connect faulty outputs to state writes. RDD lineage [4] and lineage-driven fault injection [5] motivate causal slices, while Falkirk Wheel [6] studies selective rollback. Optimistic concurrency control [7] informs conditional writes. Sagas [8] and rollback-recovery protocols [9] make clear why external effects need explicit treatment.

Validity and artifacts. AV reads simulated truth, so it is an oracle baseline. Seeds fix injection schedules, not provider sampling; results cover one task, topology, model, and pacing rule. Controlled-tool tests do not establish guarantees for remote services. Code and traces are in the artifact repository.

References

[1] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv:2406.13352*, 2024.

[2] I. Kavathekar et al. TAMAS: Benchmarking adversarial risks in multi-agent LLM systems. *ACL*, 2026.

[3] T. Li et al. A2ASecBench: A protocol-aware security benchmark for agent-to-agent multi-agent systems. *ICLR*, 2026.

[4] M. Zaharia et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *NSDI*, 2012.

[5] P. Alvaro, J. Rosen, and J. M. Hellerstein. Lineage-driven fault injection. *SIGMOD*, pp. 331–346, 2015.

[6] M. Isard and M. Abadi. Falkirk Wheel: Rollback recovery for dataflow systems. *arXiv:1503.08877*, 2015.

[7] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM TODS* 6(2):213–226, 1981.

[8] H. Garcia-Molina and K. Salem. Sagas. *SIGMOD*, 1987.

[9] E. N. Elnozahy et al. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys* 34(3):375–408, 2002.